

数学实验与数学软件课程回顾

by KOKO

预计实验3 4 5 6为重点。

实验3-插值

三种插值方法：

1. 拉格朗日插值（理论+程序）
2. 分段线性插值（理论+程序）
3. 三次样条插值（知道函数 $y=\text{spline}(x_0, y_0, x)$ ）

拉格朗日插值

理论公式：

$$L_n(x) = \sum_{i=0}^n y_i l_i(x) \quad (1)$$

$$l_i(x) = \frac{(x - x_0) \cdots (x - x_{i-1})(x - x_{i+1}) \cdots (x - x_n)}{(x_i - x_0) \cdots (x_i - x_{i-1})(x_i - x_{i+1}) \cdots (x_i - x_n)}, \quad i = 0, 1, \dots, n \quad (2)$$

$$l_i(x_j) = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases} \quad \therefore L_n(x_j) = y_j \quad (3)$$

$l_i(x)$ 称为基函数。

Note

对于函数的基本定义和计算：

```
% 定义 f(x, y) = x^2 + y^2
f = @(x, y) x.^2 + y.^2;

% 调用
res = f(3, 4); % 结果为 25
```

对于拉格朗日插值方法，需要注意的是并不能保证收敛性。（龙格现象）

分段线性插值

保证收敛但是收敛性差。(阶数为2阶)

同样使用基函数表示的公式：

$$I_n(x) = \sum_{j=0}^n y_j l_j(x) \quad (4)$$

$$l_j(x) = \begin{cases} \frac{x-x_{j-1}}{x_j-x_{j-1}}, & x_{j-1} \leq x \leq x_j \\ \frac{x-x_{j+1}}{x_j-x_{j+1}}, & x_j \leq x \leq x_{j+1} \\ 0, & \text{其它} \end{cases} \quad (5)$$

三次样条插值

四阶收敛，但是计算量较大。

编程

拉格朗日插值法：

```
% 使用基函数来计算
function y=larg(x0,y0,x)
% 先遍历x上每个点，对y(i)进行赋值
for i=1:length(x)
    y(i)=0;
    % 等于每个基函数乘以y0(j)的和
    for j=1:length(x0)
        f=1;
        % 基函数为遍历非j处之外的t的(x(i)-x0(t))/(x0(j)-x0(t))相乘
        for t=1:length(x0)
            if t~=j
                f=f*(x(i)-x0(t))/(x0(j)-x0(t));
            end
        end
        y(i)=y(i)+f*y0(j);
    end
end
```

分段插值法：

```
% 直接首先搜索到相应区间，再计算值
function y=fenduan(x0,y0,x)
for i=1:length(x)
    % 使用一个for循环找到区间
    for j=1:length(x0)-1
        if (x(i)-x0(j))*(x0(j+1)-x(i))>=0
            break
        end
    end
    % 使用此区间进行计算
    y(i)=y0(j)*(x0(j+1)-x(i))/(x0(j+1)-x0(j))+y0(j+1)*(x(i)-x0(j))/(x0(j+1)-x0(j));
end
```

三次样条插值法：

```
y=spline(x0,y0,x);
```

实验3-积分

1. 矩形公式/梯形公式/辛普森公式：知道理论
2. 积分步长的自动选取（了解概念）
3. 高斯公式（了解概念）
4. 改进的高斯公式（了解概念）

梯形公式

理论公式：

$$T_n = h \sum_{k=1}^{n-1} f_k + \frac{h}{2}(f_0 + f_n) \quad (6)$$

辛普森公式

使用连续的两个区间构造二次函数。

$$S_m = \frac{h}{3}(f_0 + f_{2m} + 4 \sum_{k=0}^{m-1} f_{2k+1} + 2 \sum_{k=1}^{m-1} f_{2k}), \quad h = \frac{b-a}{2m} \quad (7)$$

自动选取积分步长

核心：选定积分步长后，确定步长 h 满足误差 ε 。

对于复合梯形公式，其截断误差与步长 h 的平方成正比。

- 步长为 h 时：误差 $E_n = I - T_n \approx Ch^2$ 。
- 步长减半为 $h/2$ 时：误差 $E_{2n} = I - T_{2n} \approx C(\frac{h}{2})^2 = \frac{1}{4}Ch^2$ 。

通过对比可知：

$$I - T_{2n} \approx \frac{1}{4}(I - T_n) \quad (8)$$

由此推导出事后误差估计公式：

$$I - T_{2n} \approx \frac{1}{3}(T_{2n} - T_n) \quad (9)$$

这说明：相邻两次迭代结果之差的 $1/3$ 可以近似代表当前的实际误差。

我可以通过上式（通过多次迭代得到的结果）知道收敛的程度。

对于辛普森公式，以上为4改为16，3改为15。

事实上这也就是以为二者误差分别是二阶和四阶。

④ Note

这里的目的是为了避开计算常数 C （需要计算高阶导数及其极值）。

同时我们可以迭代计算：

$$T_{2n} = \frac{T_n}{2} + \frac{h}{2} \sum_{k=0}^{n-1} f_{k+1/2} \quad (10)$$

其中 $f_{k+1/2}$ 是原分点 x_k 、 x_{k+1} 的中点（记 $x_{k+1/2}$ ）的函数值。

高斯求积公式

注意到事实上，我们以上的梯形公式和辛普森公式的积分计算都是以下形式：

$$I_n = \sum_{k=1}^n A_k f(x_k) \quad (11)$$

A_k 是与 f 无关的常数。

④ Note

我们定义代数精度为：

设 $f(x) = x^k$ ，用上式计算 $I = \int_a^b f(x)dx$ 。

若对于 $k = 0, 1, \dots, m$ 都有 $I_n = I$ ，

而当 $k = m + 1$ ， $I_n \neq I$ ，则称 I_n 的代数精度为 m 。（ m 和其以下都行）

对于一个具有 n 个节点 (即 $k = 1, 2, \dots, n$) 的高斯求积公式:

$$I_n = \sum_{k=1}^n A_k f(x_k) \tag{12}$$

其代数精度为 $2n - 1$ 。

改进的高斯公式

在高斯积分的基础上，将两端的断点作为计算点：

$$G_n = A_1 f(a) + \sum_{k=2}^{n-1} A_k f(x_k) + A_n f(b) \tag{13}$$

其中 a, b 为小区间的端点， $x_2, \dots, x_{n-1}, A_1, \dots, A_n$ 为 $2n - 2$ 个参数。

代数精度可达到 $2n - 3$ 。

实验4-微分方程数值解

1. 实例
2. 常用的数值算法：

▪ (改进) 欧拉方法 (理论+编程)

▪ 龙格库塔方法 (编程)
3. 数值算法的收敛性 (掌握原理)
4. 编程

实例

$$\begin{cases} \frac{dx}{dt} = rx - axy & (\text{食饵数量变化} = \text{自主增长} - \text{被吃掉}) \\ \frac{dy}{dt} = -dy + bxy & (\text{捕食者数量变化} = \text{自然死亡} + \text{捕食增益}) \end{cases} \tag{14}$$

感觉建模题会出微分方程的解，只有这里有实例。

欧拉方法 (估计考这个)

向前欧拉方法：

$$y_{n+1} = y_n + hf(x_n, y_n) \tag{15}$$

先后欧拉方法：

$$y_{n+1} = y_n + hf(x_{n+1}, y_{n+1}) \tag{16}$$

以上二者得到梯形公式：

$$y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, y_{n+1})], n = 0, 1, \dots \quad (17)$$

由于以上为一个隐式方程，我们提出以下改进欧拉方法：

$$\bar{y}_{n+1} = y_n + hf(x_n, y_n) \quad (18)$$

$$y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, \bar{y}_{n+1})], n = 0, 1, \dots \quad (19)$$

龙格库塔法（ L 级情况）：

$$\begin{cases} y_{n+1} = y_n + h \sum_{i=1}^L \lambda_i k_i \\ k_1 = f(x_n, y_n) \\ k_2 = f(x_n + c_2 h, y_n + c_2 h k_1) \\ \dots \\ k_i = f(x_n + c_i h, y_n + c_i h \sum_{j=1}^{i-1} a_{ij} k_j), \quad i = 3, 4, \dots, L \end{cases} \quad (20)$$

其中 λ_i 、 c_i 、 a_{ij} 满足

$$\sum_{i=1}^L \lambda_i = 1, 0 \leq c_i \leq 1, \sum_{j=1}^{i-1} a_{ij} = 1 \quad (21)$$

一般的4级4阶情况：

$$\begin{cases} y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \\ k_1 = f(x_n, y_n) \\ k_2 = f(x_n + h/2, y_n + hk_1/2) \\ k_3 = f(x_n + h/2, y_n + hk_2/2) \\ k_4 = f(x_n + h, y_n + hk_3) \end{cases} \quad (22)$$

① Note

这里的级数指的是计算 f 的次数，阶数是算法精度。

① Note

我们来对于误差做一个介绍：

整体截断误差：

从初值 $x(t_0) = x_0$ 出发，由单步法逐步计算得 t_{n+1} 处的数值解 x_{n+1} ，则 $e_{n+1} = x(t_{n+1}) - x_{n+1}$ 称为在点 t_{n+1} 处的整体截断误差。

局部截断误差：

若第 n 步的数值计算没有误差，即 $x_n = x(t_n)$ ，由单步法计算出 $\tilde{x}_{n+1}$ ，则 $T_{n+1} = x(t_{n+1}) - \tilde{x}_{n+1}$ 称为在点 t_{n+1} 处的局部截断误差。

算法精度：

如果给定的数值算法的局部截断误差为 $T_{n+1} = \mathcal{O}(h^{p+1})$ ，则称该算法具有 p 阶精度。

误差分析

对于欧拉方法的误差

精确解的泰勒展开：

$$y(x_{n+1}) = y(x_n) + hy'(x_n) + \frac{h^2}{2}y''(\xi)$$
(23)

向前欧拉数值解：

$$y_{n+1} = y(x_n) + hy'(x_n)$$
(24)

局部截断误差定义与计算：

$$T_{n+1} = y(x_{n+1}) - y_{n+1} = \frac{h^2}{2}y''(\xi)$$
(25)

误差量级结论：

$$T_{n+1} = O(h^2)$$
(26)

可以得到的一个结论是：

方法名称	局部截断误差	整体误差 (Global Error)	精度 (阶数)
向前欧拉公式	$O(h^2)$	$O(h)$	1 阶
向后欧拉公式	$O(h^2)$	$O(h)$	1 阶
梯形公式	$O(h^3)$	$O(h^2)$	2 阶
改进欧拉公式	$O(h^3)$	$O(h^2)$	2 阶
经典龙格-库塔公式	$O(h^5)$	$O(h^4)$	4 阶

事实上，欧拉方法就是1级1阶的龙格库塔方法。改进欧拉方法就是2级2阶的龙格库塔方法。

对于整体截断误差，我们的误差阶数会比阶段误差精度小一位，也就是恰恰等于算法精度。

编程

龙格库塔方法（同时满足默认相对误差 10^{-3} ，绝对误差 10^{-6} ，计算步长会按照误差要求调整）。：

```
% 三级二阶,其中t=ts,x0为初始值, 舒服t=ts时的x值
[t,ts]=ode23(@f,ts,x0)

% 对于f,使用function定义
function dx=f(x,t)
    dx=x+t;
end
```

自己编写欧拉方法：

```
% 定义函数
function dx=f(x,t)
    for i=1:length(x)
        dx(i)=x(i)+t;
    end
end

% 编写一个欧拉方法的函数，给一个区间，输出等距分割后的x和y的值
function [tt,x]=euler(f,t,x0,n)
    h=(t(2)-t(1))/n;
    tt=t(1):h:t(2);
    x(1)=x0;
    for i=2:n+1
        x(i)=x(i-1)+h*f(x(i-1),tt(i-1));
    end
end

% 编写一个改进欧拉方法的函数，给一个区间，输出等距分割后的x和y的值
function [tt,x]=eulerimprove(f,t,x0,n)
    h=(t(2)-t(1))/n;
    tt=t(1):h:t(2);
    x(1)=x0;
    for i=2:n+1
        x(i)=x(i-1)+h*f(x(i-1),tt(i-1));
        x(i)=x(i-1)+h*(f(x(i-1),tt(i-1))+f(x(i),tt(i)))/2;
    end
end
```

实验5-线性方程组的数值解法

1. 两类数值方法（代码+理论）：
 - 直接方法
 - 迭代方法
2. 迭代法的收敛性判断（代码+理论）
3. 条件数与误差估计
4. 编程

两类解法

线性方程组的一般形式和两类解法：

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \dots\dots\dots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases} \quad (27)$$

或

$$AX = b \quad (28)$$

直接法：

经过有限次算术运算求出精确解（实际上由于有舍入误差只能得到近似解）：高斯（*Gauss*）消元法及与它密切相关的矩阵LU分解

迭代法：

从初始解出发，根据设计好的步骤用逐次求出的近似解逼近精确解——雅可比（*Jacobi*）迭代法和高斯—塞德尔（*Gauss—Seidel*）迭代法

直接法

过程就是简单消元，需要注意的是我们尽量选取最大的那个元素，来规避舍入误差。

或者可以使用更加快捷的LU分解：

$$PA = LU \tag{29}$$

其中 P ：置换矩阵（记录行交换）。

迭代法

雅可比迭代公式：

$$x^{(k+1)} = D^{-1}(L + U)x^k + D^{-1}b \tag{30}$$

高斯赛德尔迭代公式：

$$Dx^{(k+1)} = (D - L)^{-1}Ux^k + (D - L)^{-1}b \tag{31}$$

收敛的充要条件：谱半径小于1。

收敛的充分条件：

- 1. 严格对角占优，则雅可比和高斯赛德尔都收敛。
- 2. 若 A 对称正定，则高斯赛德尔迭代收敛
- 3. 若矩阵的范数小于1，则迭代公式收敛，且范数越小收敛越快。

④ Note

使用结论：谱半径为范数集合的下界。

编程

雅可比迭代:

```
% 雅可比迭代
function [inter,x]=yakebi(A,b,x0,maxinter,tol)
    x=x0;
    d=diag(A);
    R=diag(d)-A;
    inter=0;
    while inter<maxinter
        x_new=(R*x+b)./d;
        if norm(x_new-x,inf)<=tol
            inter=inter+1;
            break
        end
        x=x_new;
        inter=inter+1;
    end
end
```

高斯赛德尔迭代:

```
% 高斯赛德尔迭代
function [inter,x]=gaosisaideer(A,b,x0,maxinter,tol)
    x=x0;
    % 取下三角 (含对角)
    R=tril(A);
    D=R-A;
    inter=0;
    while inter<maxinter
        x_new=R\D*x+R\b;
        if norm(x_new-x,inf)<=tol
            inter=inter+1;
            break
        end
        x=x_new;
        inter=inter+1;
    end
end
```

① Note

使用稀疏矩阵 (估计也要考):

```
% 定义稀疏矩阵
n=10;
A=sparse(1:n,1:n,1,n,n);
B=sparse(2:n,1:n-1,1,n,n);
C=A+B+B';
% 重新转为满矩阵
D=full(C)
```

一些需要知道的函数：

函数	核心功能	输入类型	输出类型
lu	LU 分解	矩阵	两个矩阵 (L, U)
chol	Cholesky 分解	正定矩阵	一个上三角矩阵
norm	计算范数	向量/矩阵	标量
eig	特征分解	方阵	向量(特征值)或矩阵
diag	提取/构造对角线	矩阵/向量	向量/矩阵

实验6-非线性方程求解

1. 非线性方程 $f(x)$ 的数值解法：
 - 迭代方法的基本原理
 - 牛顿法（理论）
 - 拟牛顿法（理论+编程）
2. 推广到解非线性方程组（牛顿迭代相关，收敛性判断）
3. 分岔和混沌（了解，那就是不考喽）

非线性方程的基本解法

基本方法有：图形法和二分法。

迭代法：

将原方程 $f(x) = 0$ 改写成容易迭代的形式 $x = \varphi(x)$ ，选合适的初值 x_0 ，进行迭代：

$$x_{k+1} = \varphi(x_k) \quad (k = 0, 1, 2, \cdots) \tag{32}$$

局部收敛性：只要 $\varphi(x)$ 在 x^* 的一个邻域连续且 $|\varphi'(x^*)| < 1$ ，则对于该邻域内的任意初值 x_0 ，序列 $\{x_k\}$ 就收敛于 x^* 。

牛顿切线法

首先得到

$$f(x) = f(x_k) + f'(x_k)(x - x_k) \tag{33}$$

然后进而得到

设 $f'(x_k) \neq 0$ ，用 x_{k+1} 代替右端的 x ，由 $f(x) = 0$ 得

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}, \quad \text{即令} \quad \varphi(x) = x - \frac{f(x)}{f'(x)} \quad (34)$$

若 x^* 为单根:

$$f(x^*) = 0, f'(x^*), f''(x^*) \neq 0 \quad (35)$$

$$\Rightarrow \varphi'(x^*) = 0, \varphi''(x^*) \neq 0 \quad (36)$$

牛顿切线法 2 阶收敛。

若 x^* 为重根:

$$f(x^*) = f'(x^*) = 0 \Rightarrow \varphi'(x^*) \neq 0 \quad (37)$$

牛顿切线法 1 阶收敛。

Note

重数越高, 收敛越慢。

解非线性方程组的牛顿方法

我们可以将以上结论推广到方程组上:

$$F'(x) = \left[\frac{\partial f_i}{\partial x_j} \right]_{n \times n} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix} \quad (38)$$

我们由迭代方程:

$$F(x^{k+1}) = F(x^k) + F'(x^k)(x^{k+1} - x^k) \quad (39)$$

$$x^{k+1} = x^k - [F'(x^k)]^{-1} F(x^k) \quad (40)$$

拟牛顿法

为了不计算导数, 我们引入以下迭代方式:

解方程 $f(x) = 0$ 的牛顿割线法

$$f(x^{k+1}) = f(x^k) + a^k(x^{k+1} - x^k) \quad (41)$$

$$x^{k+1} = x^k - \frac{f(x^k)}{a^k} \quad (42)$$

$$a^k = \frac{f(x^k) - f(x^{k-1})}{x^k - x^{k-1}} \quad (43)$$

而对于方程组:

我们要求 A^k 满足

$$A^k(x^k - x^{k-1}) = F(x^k) - F(x^{k-1}) \quad (44)$$

但是矩阵 A^k (n^2 个未知数) 不能由这样的 n 个方程确定。

用迭代方法 $A^k = A^{k-1} + \Delta A^{k-1}$ (ΔA 有不同的构造) 计算 A^k 。

再求：

$$x^{k+1} = x^k - (A^k)^{-1} F(x^k) \quad (45)$$

Note

这里 Δ 的构造不具体阐述，较为复杂。

编程

牛顿迭代法：

```
% 牛顿法求零点
function [inter,x]=neton(A,J,x0,maxinter,tol)
    x=x0;
    inter=0;
    while inter<maxinter
        x_new=x-J(x)\A(x);
        if norm(x_new-x,inf)<=tol
            inter=inter+1;
            break
        end
        x=x_new;
        inter=inter+1;
    end
end
```

单变量方程：

```
% 单变量方程
[x,fv,ef,out]=fzero(@f,x0,opt,P1,P2...)
% --- 输入参数 (Input Arguments) ---
% @f    : 函数句柄。指向待求解的方程 f(x)。可以是匿名函数 @(x) x^2-2 或函数文件名。
% x0    : 初始值。
% opt    : 选项结构体。使用 optimset 函数创建（如设置精度 'TolX' 或显示过程 'Display'）。
% P1,P2: 附加参数。传递给函数 f 的额外常数（注：现代 MATLAB 更推荐使用匿名函数传递参数）。

% --- 输出参数 (Output Arguments) ---
% x      : 求解出的根。即满足 f(x) 约等于 0 的点。
% fv     : (Function Value) 函数在根 x 处的取值。理想情况下应接近于 0。
% ef     : (Exit Flag) 退出标志。用于判断计算是否成功：
%         1 : 找到根；
%         0 : 超过限制；
%        -1 : 算法被输出函数中断；
%        -3 : 在搜索区间内没有找到符号改变的点；

% out    : (Output Structure) 包含优化信息的结构体。
%         - out.iterations: 迭代次数；
%         - out.funcCount: 函数计算次数；
```

```
% - out.algorithm: 所使用的算法（通常是 Brent 算法）。
```

多变量方程组：

```
[x,fv,ef,out,jac]=fsolve(@f,x0,opt,P1,P2...)
```

实验7-无约束优化

1. 优化问题的最优解条件
2. 无约束优化的基本方法：
 1. 梯度法(一阶)
 2. 牛顿法 (二阶)
 3. 拟牛顿法 (超线性, 介于1 2, 没有介绍, 应该不考)
3. 非线性最小二乘法
4. 优化工具箱 (了解, 那就是不考喽)

最优解

最优解条件的充要条件：

$$\nabla f(x^*) = 0, \nabla^2 f(x^*) > 0 \quad \nabla^2 f = \left[\frac{\partial^2 f}{\partial x_i \partial x_j} \right]_{n \times n} \quad (46)$$

以上这个二次阶的矩阵称为 *Hessian* 矩阵。

一个基本想法是使用点之间的迭代。

搜索方向的选择

最速下降法：

搜索方向为：

$$d^k = -\nabla f(x^k) \quad (\text{负梯度方向}) \quad (47)$$

最优解附近收敛缓慢。

Newton 方法：

$$f(x^{k+1}) = f(x^k + d) = f(x^k) + \nabla f^T(x^k)d + \frac{1}{2}d^T \nabla^2 f(x^k)d \quad (48)$$

求 d 使 $f(x^{k+1})$ 极小 $\Rightarrow$ 右端对 d 导数为 0 $\Rightarrow \nabla f(x^k) + \nabla^2 f(x^k)d = 0$

牛顿方程 $\nabla^2 f(x^k)d = -\nabla f(x^k)$

牛顿方向

$$d^k = -(\nabla^2 f(x^k))^{-1} \nabla f(x^k) \quad (49)$$

迭代格式 $x^{k+1} = x^k - (\nabla^2 f(x^k))^{-1} \nabla f(x^k)$

特点：局部2阶收敛；需计算Hessian阵，它可能病态或不正定。

事实上，我们就是等同在求解

$$\nabla f(x^k) = 0 \quad (50)$$

我们就将问题转化为一个求零点的问题。

编程

```
function [x_opt, iter_history] = newton_method(f_grad, f_hessian, x0, tol, max_iter)
    % 牛顿法求解无约束优化问题
    % 输入：
    %   f_grad: 梯度函数句柄
    %   f_hessian: Hessian矩阵函数句柄
    %   x0: 初始点
    %   tol: 收敛容差（梯度范数）
    %   max_iter: 最大迭代次数

    x = x0;
    iter_history = x; % 记录迭代轨迹

    fprintf('迭代次数\t 梯度范数\n');

    for k = 1:max_iter
        g = f_grad(x); % 计算梯度
        H = f_hessian(x); % 计算Hessian矩阵

        grad_norm = norm(g);
        fprintf('%d\t\t %e\n', k-1, grad_norm);

        % 检查收敛性
        if grad_norm < tol
            fprintf('算法收敛。 \n');
            x_opt = x;
            return;
        end

        % 求解牛顿方程 H*d = -g
        % 注意：如果H不是正定的，牛顿方向可能不是下降方向
        d = H \ (-g);

        % 更新变量
        x = x + d;
        iter_history = [iter_history, x];
    end
end
```

```
x_opt = x;  
warning('达到最大迭代次数，可能未完全收敛。');  
end
```

实验8-线性规划

1. 基本原理和算法

- 二维线性规划的图解法
- 一般线性规划的单纯形算法（标准形比较重要）
- 一般线性规划的内点算法
- 对偶问题

2. 编程 ([x,fval,exitflag,output,lambda]=linprog(c,A1,b1,A2,b2,v1,v2,x0,opt))

标准形

标准形：

$$\min \quad z = c^T x \quad (51)$$

$$s. t. \quad Ax = b, x \geq 0 \quad (52)$$

$$A \in R^{m \times n}, m \leq n \quad (53)$$

变形过程如下：

$$\max z = 3x_1 + x_2 \quad (54)$$

$$s. t. \quad x_1 - x_2 \geq -2 \quad (55)$$

$$x_1 - 2x_2 \leq 2 \quad (56)$$

$$3x_1 + 2x_2 \leq 14 \quad (57)$$

$$x_1, x_2 \geq 0 \quad (58)$$

可以转化为

$$\min z = -3x_1 - x_2 + 0 \cdot x_3 + 0 \cdot x_4 + 0 \cdot x_5 \quad (59)$$

$$s. t. \quad -x_1 + x_2 + x_3 = 2, \quad x_3 \geq 0 \quad (60)$$

$$x_1 - 2x_2 + x_4 = 2, \quad x_4 \geq 0 \quad (61)$$

$$3x_1 + 2x_2 + x_5 = 14, \quad x_5 \geq 0 \quad (62)$$

$$x_1, x_2 \geq 0 \quad (63)$$

编程

```
[x, fval, exitflag, output, lambda] = linprog(c, A1, b1, A2, b2, v1, v2, x0, opt)

%% --- 输入参数 (Input Arguments) ---
c      % 目标函数的系数向量（列向量）。
A1     % 不等式约束矩阵 A1 * x <= b1。
b1     % 不等式约束的右侧向量。
A2     % 等式约束矩阵 A2 * x = b2。
b2     % 等式约束的右侧向量。
v1     % 变量 x 的下界向量（Lower Bound），若无则设为 []。
v2     % 变量 x 的上界向量（Upper Bound），若无则设为 []。
x0     % 初始解向量（通常可省略，部分算法会参考此值）。
opt    % 优化选项，使用 optimoptions('linprog',...) 设置（如指定算法 'dual-simplex'）。

%% --- 输出参数 (Output Arguments) ---
x      % 求解得到的最优解向量。
fval   % 目标函数在最优解处的取值：fval = c' * x。
exitflag % 退出标志，用于判断求解状态：
        % 1 : 函数收敛于解 x。
        % 0 : 达到最大迭代次数。
        % -2 : 问题无解（Infeasible）。
        % -3 : 问题无界（Unbounded）。
output % 包含优化过程信息的结构体（如迭代次数、算法类型等）。
lambda % 包含解处的拉格朗日乘子（Lagrange multipliers）的结构体：
        % .lower : 下界 v1 对应的乘子。
        % .upper : 上界 v2 对应的乘子。
        % .ineqlin : 线性不等式约束对应的乘子。
        % .eqlin : 线性等式约束对应的乘子。
```

实验9-非线性规划

有效集法通常用于求解**二次规划** (Quadratic Programming, QP) 问题。其标准数学形式为：

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) = \frac{1}{2}x^T Gx + c^T x \\ \text{s.t.} \quad & a_j^T x \leq b_j, \quad j = 1, 2, \dots, l \\ & a_j^T x = b_j, \quad j = l + 1, \dots, m \end{aligned} \quad (64)$$

其中 G 是正定或半正定矩阵（保证问题是凸的）。

2. 核心理论：KKT 条件 (Optimality Conditions)

如果 x^* 是上述问题的最优解，则必须存在一组拉格朗日乘子 λ_j ，满足以下 **KKT 条件**：

1. 平稳性 (Stationarity): 目标函数与约束函数的梯度平衡

$$\nabla f(x^*) + \sum_{j=1}^m \lambda_j \nabla g_j(x^*) = 0 \quad (65)$$

2. **可行性 (Primal Feasibility):** x^* 必须满足所有原始约束。
3. **非负性 (Dual Feasibility):** 对于不等式约束，乘子 $\lambda_j \geq 0$ 。
4. **互补松弛性 (Complementary Slackness):** $\lambda_j(a_j^T x^* - b_j) = 0, \quad \forall j$ 这意味着：如果约束没达到边界（不起作用），乘子必须为 0；如果乘子不为 0，约束必须在边界上（起作用）。

编程

```
[x, fv, ef, out, lag, grad, hess] = fmincon(@f, x0, A1, b1, A2, b2, v1, v2, @c, opt, P1, P2, ...)
```

主要参数含义：

- `@f`: 目标函数。
- `x0`: 初始搜索点。
- `A1, b1`: 线性不等式约束 $A_1x \leq b_1$ 。
- `A2, b2`: 线性等式约束 $A_2x = b_2$ 。
- `v1, v2`: 变量的上下界 (Lower/Upper Bounds) 。
- `@c`: 非线性约束函数（包含不等式和等式）。
- `lag`: 返回的**拉格朗日乘子** (Lagrange multipliers)，对应上述理论中的 λ 。

实验10-整数规划

1. 分支定界
2. 动态规划

考了那没话说。

实验11-数据的统计与分析

1. 统计基本概念
2. 用随机模拟计算数值积分

阅读直方图（一般是左闭右开）

简单的统计分析函数：

命令	名称	输入	输出	注意事项
<code>[n,y]=hist(x,k)</code>	频数表	x : 原始数据行向量 k : 等分区间数	n : 频数行向量 y : 区间中点行向量	<code>[n,y]=hist(x)</code> 中 k 取缺省值 10
<code>hist(x,k)</code>	直方图	同上	直方图	同上

命令	名称	输入	输出	注意事项
mean(x)	均值	x : 原始数据行向量	均值	
median(x)	中位数	同上	中位数	
range(x)	极差	同上	极差	
std(x)	标准差	同上	标准差 s	std(x,1): 公式中 $n - 1$ 改成 n
var(x)	方差	同上	方差 s^2	var(x,1): 同上
skewness(x)	偏度	同上	偏度 g_1	
kurtosis(x)	峰度	同上	峰度 g_2	

偏度：

定义：数据分布的三阶标准化矩。 $g_1 = E\left[\left(\frac{X-\mu}{\sigma}\right)^3\right]$

峰度：

定义：数据分布的四阶标准化矩。 $g_2 = E\left[\left(\frac{X-\mu}{\sigma}\right)^4\right]$

数值含义：

- $g_2 = 3$ ：正态分布的峰度为 3。
- $g_2 > 3$ **(尖峰)**：分布中心更陡峭，且拥有更厚的尾部（厚尾），意味着极端值出现的概率比正态分布大。
- $g_2 < 3$ **(平峰)**：分布中心较平坦，尾部较薄。

分布类型	概率密度 (pdf)	分布函数 (cdf)	逆概率分布 (inv)	均值与方差 (stat)	随机数生成 (rnd)
均匀分布 (<i>unif</i>)	unifpdf	unifcdf	unifinv	unifstat	unifrnd
指数分布 (<i>exp</i>)	exppdf	expcdf	expinv	expstat	exprnd
正态分布 (<i>norm</i>)	normpdf	normcdf	norminv	normstat	normrnd
χ^2分布 (<i>chi2</i>)	chi2pdf	chi2cdf	chi2inv	chi2stat	chi2rnd
t分布 (<i>t</i>)	tpdf	tcdf	tinv	tstat	trnd
F分布 (<i>f</i>)	fpdf	fcdf	finv	fstat	frnd
二项分布 (<i>bin</i>)	binopdf	binocdf	binoinv	binostat	binornd
泊松分布 (<i>poiss</i>)	poisspdf	poisscdf	poissinv	poissstat	poissrnd

实验12-统计推断

1. 参数估计（点估计，区间估计）
2. 假设检验

编程

假设检验（单样本or检查差和均值）：

```
[h, p, ci, stats] = ttest(x);           % 检验 x 的均值是否为 0
[h, p, ci, stats] = ttest(x, m);       % 检验 x 的均值是否为 m
[h, p, ci, stats] = ttest(x, y);       % 检验配对样本 x 和 y 的均值是否相等
```

假设检验（双样本）：

```
[h, p, ci, stats] = ttest2(x, y);      % 默认假设两组数据方差相等
[h, p, ci, stats] = ttest2(x, y, 'Vartype', 'unequal'); % 假设方差不等 (Welch's T-test)
```

已知总体方差：

```
[h, sig, ci, zval] = ztest(x, mu, sigma, alpha, tail)
```

输入参数 x 是样本 (n 维数组)， mu 是 H_0 中的 μ_0 ， $sigma$ 是总体标准差 σ ， $alpha$ 是显著性水平 α （默认时设定为 0.05）， $tail$ 是对双侧检验和两个单侧检验的标识，用备选假设 H_1 确定： H_1 为 $\mu \neq \mu_0$ 时令 $tail = 0$ （可默认）； H_1 为 $\mu > \mu_0$ 时令 $tail = 1$ ； H_1 为 $\mu < \mu_0$ 时令 $tail = -1$ 。

输出参数 $h = 0$ 表示接受 H_0 ， $h = 1$ 表示拒绝 H_0 ， sig 是在假设 H_0 下的概率 $P\{|Z| \geq |z|\}$ ，其中 $Z \sim N(0, 1)$ ， $z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}}$ ， ci 给出 μ_0 的置信区间， $zval$ 是样本统计量 z 的值。

无论哪个函数，返回值的含义一致：

- **h**：假设检验结果。若 $h=1$ 表示拒绝原假设（有显著差异）； $h=0$ 表示接受原假设。
- **p**：P 值。通常 $p < 0.05$ 认为具有统计学显著性。
- **ci**：均值差的 95% 置信区间。
- **stats**：包含统计量信息的结构体（如 `tstat` 统计量值、`df` 自由度等）。

实验13-回归统计分析

1. 一元线性回归—参数估计（点估计，区间估计）和模型有效性检验
2. 多元线性回归
3. 非线性回归（`regress`）

一元线性回归模型

一元线性回归模型

$$y = \beta_0 + \beta_1 x + \varepsilon \quad (66)$$

$x \sim$ 自变量, β_0 、 $\beta_1 \sim$ 回归系数。

$\varepsilon \sim$ 随机变量 (除 x 外影响 y 的随机因素的总和)。

四个基本假设:

1. 独立性: 对于不同的 x , y 相互独立
2. 线性性: y 的期望是 x 的线性函数
3. 齐次性: 对于不同的 x , y 的方差是常数
4. 正态性: 对于给定的 x , y 服从正态分布

其中 ε 是相互独立的、期望为 0、方差为 σ^2 、正态分布的随机变量, 即 $\varepsilon \sim N(0, \sigma^2)$, ε 称 (随机) 误差。

最小二乘估计

数据 x_i 、 y_i ($i = 1, \dots, n$) 代入 $y = \beta_0 + \beta_1 x + \varepsilon \Rightarrow y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$

误差平方和

$$Q(\beta_0, \beta_1) = \sum_{i=1}^n \varepsilon_i^2 = \sum_{i=1}^n [y_i - (\beta_0 + \beta_1 x_i)]^2 \quad (67)$$

$$\frac{\partial Q}{\partial \beta_0} = 0, \frac{\partial Q}{\partial \beta_1} = 0 \Rightarrow \hat{\beta}_1 = \frac{s_{xy}}{s_{xx}}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} \quad (68)$$

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i, s_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2, s_{xy} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) \quad (69)$$

直线 $y = \hat{\beta}_0 + \hat{\beta}_1 x$ 通过 x_i 、 y_i 的均值点 $(\bar{x}, \bar{y})$

回归模型的有效性检验

1. t 检验 (t-Test)

用途: 主要用于检验单个回归系数的显著性。

- **统计量:** $|t| = \left| \frac{\hat{\beta}_1}{se(\hat{\beta}_1)} \right| = \left| \frac{\hat{\beta}_1 \sqrt{S_{xx}}}{s} \right|$
- **判定:** 若 $|t| > t_{n-2}(1 - \alpha/2)$, 则拒绝 H_0 , 说明 x 对 y 的线性影响显著。
- **特点:** 在多元回归中, t 检验可以针对每一个自变量进行单独考核。

2. F 检验 (F-Test)

用途: 从方差分析 (ANOVA) 的角度检验回归方程的整体显著性。

- **原理：**将总偏差平方和 S 分解为回归平方和 U （由 x 解释的部分）和残差平方和 Q （未解释的部分）。
- **统计量：** $F = \frac{U/1}{Q/(n-2)} \sim F(1, n-2)$
- **判定：**若 $F > F_{1-\alpha}(1, n-2)$ ，则拒绝 H_0 ，模型有效。
- **联系：**在简单线性回归（只有一个自变量）中， $F = t^2$ ，两者的结论是完全一致的。

3. 相关系数检验 / 判定系数 (R^2)

用途：衡量模型对数据的**拟合优度**（Goodness of Fit）。

- **定义：** $R^2 = \frac{U}{S} = 1 - \frac{Q}{S}$
- **意义：** R^2 越接近 1，表示回归线对观测值的代表性越好。
- **检验方式：**虽然 R^2 本身是一个描述性指标，但可以通过相关系数 r 的显著性检验来判断。对于简单线性回归， R^2 就是样本相关系数 r 的平方。

④ Note

计算基础离差：

$$S_{xx} = \sum (x_i - \bar{x})^2 \quad S_{yy} = \sum (y_i - \bar{y})^2 \quad S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y})$$

计算回归系数：

$$\hat{\beta}_1 = \frac{S_{xy}}{S_{xx}}$$

计算平方和：

- $U = \hat{\beta}_1 S_{xy}$
- $Q = S_{yy} - U$

得出最终指标：

- $s = \sqrt{Q/(n-2)}$
- $R^2 = U/S_{yy}$